RAJALAKSHMI ENGINEERING COLLEGE

(Autonomous)

RAJALAKSHMI NAGAR, THANDALAM, CHENNAI-602105

EXPT NO: 4	**DATE: ________________**

A PYTHON PROGRAM TO SOLVE AN IPP USING THE BRANCH AND BOUND METHOD

AIM:

To solve Integer Programming Problems (IPP) using the Branch and Bound technique in Python, obtaining optimal integer solutions.

PROCEDURE:

1. Formulate the Integer Programming Problem with objective and constraints.
2. Implement Branch and Bound using a queue to explore sub-problems.
3. At each node, solve the LP relaxation using linprog.
4. If the solution has non-integer values, branch on the first fractional variable.
5. Track and update the best integer feasible solution found.

PROBLEM 1: Maximize Z = 4x + 3y s.t. 2x + y ≤ 14, x + 2y ≤ 14, x,y ≥ 0 integers

CODE

```
import numpy as np
from scipy.optimize import linprog
from queue import Queue
def solve_lp(c, A, b, bounds):
return linprog(c, A_ub=A, b_ub=b, bounds=bounds, method='highs')
def branch_bound(c, A, b, bounds, maximize=True):
Q = Queue()
Q.put((c, A, b, bounds))
best_val = float('-inf') if maximize else float('inf')
best_sol = None
while not Q.empty():
cp = Q.get()
res = solve_lp(*cp)
if not res.success: continue
sol = res.x
val = -res.fun if maximize else res.fun
if all(np.isclose(sol, np.round(sol))):
if (maximize and val > best_val) or (not maximize and val < best_val):
best_val, best_sol = val, sol
else:
for i in range(len(sol)):
if not np.isclose(sol[i], round(sol[i])):
lb = list(cp[3]); ub = list(cp[3])
lb[i] = (lb[i][0], np.floor(sol[i]))
ub[i] = (np.ceil(sol[i]), ub[i][1])
```

```
Q.put((cp[0],cp[1],cp[2],lb))
Q.put((cp[0],cp[1],cp[2],ub))
break
return np.round(best_sol), best_val
c = [-4, -3]; A = [[2,1],[1,2]]; b = [14,14]
bounds = [(0, None), (0, None)]
sol, val = branch_bound(c, A, b, bounds, maximize=True)
print("Optimal solution:", sol)
print("Optimal value Z =", val)
```

OUTPUT

```
Optimal solution: [4. 5.]
Optimal value Z = 31.0
```

PROBLEM 2: Maximize Z = 5x + 4y s.t. x + y ≤ 5, 10x + 6y ≤ 45, x,y ≥ 0 integers

CODE

```
c = [-5, -4]; A = [[1,1],[10,6]]; b = [5,45]
bounds = [(0, None), (0, None)]
sol, val = branch_bound(c, A, b, bounds, maximize=True)
print("Optimal solution:", sol)
print("Optimal value Z =", val)
```

OUTPUT

```
Optimal solution: [3. 2.]
Optimal value Z = 23.0
```

PROBLEM 3: Minimize Z = 2x + 3y s.t. x + y ≥ 4, x + 3y ≥ 6, x,y ≥ 0 integers

CODE

```
c = [2, 3]
A = [[-1, -1], [-1, -3]]
b = [-4, -6]
bounds = [(0, None), (0, None)]
sol, val = branch_bound(c, A, b, bounds, maximize=False)
print("Optimal solution:", sol)
print("Minimum Z =", val)
```

OUTPUT

```
Optimal solution: [3. 1.]
Minimum Z = 9.0
```

RESULT:

Thus, the Integer Programming Problem using Branch and Bound method in Python has been implemented and executed successfully.